In [2]:
```
#Name: Hemant Marotkar ,UID:- 23015002

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

# 1. Load IDS dataset
DATASET_PATH = "ids_dataset.csv"
df = pd.read_csv(DATASET_PATH)
print("Dataset shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

# 2. Separate input features and target label
X = df.drop("label", axis=1)
y = df["label"]

# 3. Identify categorical and numerical columns
categorical_features = ["protocol_type", "service", "flag"]
numerical_features = [col for col in X.columns if col not in categorical_features]

# 4. Preprocessing:
#    - OneHotEncoder converts text features into numbers
#    - StandardScaler normalizes numerical features
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
        ("num", StandardScaler(), numerical_features),
    ]
)

# 5. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y,
)

# 6. Create Decision Tree model pipeline
decision_tree_model = Pipeline(
    steps=[
        ("preprocessing", preprocessor),
        ("classifier", DecisionTreeClassifier(random_state=42, max_depth=5)),
    ]
)

# 7. Create Random Forest model pipeline
```

```
random_forest_model = Pipeline(
    steps=[
        ("preprocessing", preprocessor),
        (
            "classifier",
            RandomForestClassifier(
                n_estimators=100,
                random_state=42,
                max_depth=5,
            ),
        ),
    ]
)

# 8. Train both models
decision_tree_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)

# 9. Make predictions on test data
dt_predictions = decision_tree_model.predict(X_test)
rf_predictions = random_forest_model.predict(X_test)

# 10. Evaluate both models
dt_accuracy = accuracy_score(y_test, dt_predictions)
rf_accuracy = accuracy_score(y_test, rf_predictions)

print("\n================ MODEL COMPARISON ================")
print(f"Decision Tree Accuracy : {dt_accuracy:.2f}")
print(f"Random Forest Accuracy : {rf_accuracy:.2f}")

print("\nDecision Tree Confusion Matrix:")
print(confusion_matrix(y_test, dt_predictions))

print("\nRandom Forest Confusion Matrix:")
print(confusion_matrix(y_test, rf_predictions))

print("\nDecision Tree Classification Report:")
print(classification_report(y_test, dt_predictions))

print("\nRandom Forest Classification Report:")
print(classification_report(y_test, rf_predictions))

# 11. Simple conclusion
if rf_accuracy > dt_accuracy:
    print("\nConclusion: Random Forest performed better than Decision Tree.")
elif rf_accuracy < dt_accuracy:
    print("\nConclusion: Decision Tree performed better than Random Forest.")
else:
    print("\nConclusion: Both models gave the same accuracy on this sample dataset.")
```

```
Dataset shape: (40, 17)

First 5 rows:
   duration protocol_type service flag  src_bytes  dst_bytes  count  \
0         0           tcp    http   SF        181       5450      8
1         0           tcp    http   SF        239        486      8
2         0           tcp    http   SF        235       1337      8
3         0           tcp    http   SF        219       1337      6
4         0           tcp    http   SF        217       2032      6

   srv_count  serror_rate  srv_serror_rate  same_srv_rate  diff_srv_rate  \
0          8            0                0            1.0            0.0
1          8            0                0            1.0            0.0
2          8            0                0            1.0            0.0
3          6            0                0            1.0            0.0
4          6            0                0            1.0            0.0

   dst_host_count  dst_host_srv_count  dst_host_same_srv_rate  \
0               9                   9                     1.0
1              19                  19                     1.0
2              29                  29                     1.0
3              39                  39                     1.0
4              49                  49                     1.0

   dst_host_serror_rate   label
0                   0.0  normal
1                   0.0  normal
2                   0.0  normal
3                   0.0  normal
4                   0.0  normal

================ MODEL COMPARISON ================
Decision Tree Accuracy : 1.00
Random Forest Accuracy : 1.00

Decision Tree Confusion Matrix:
[[5 0]
 [0 7]]

Random Forest Confusion Matrix:
[[5 0]
 [0 7]]

Decision Tree Classification Report:
              precision    recall  f1-score   support

      attack       1.00      1.00      1.00         5
      normal       1.00      1.00      1.00         7

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12


Random Forest Classification Report:
              precision    recall  f1-score   support
```

```
              precision    recall  f1-score   support

      attack       1.00      1.00      1.00         5
      normal       1.00      1.00      1.00         7

    accuracy                           1.00        12
   macro avg       1.00      1.00      1.00        12
weighted avg       1.00      1.00      1.00        12
```

Conclusion: Both models gave the same accuracy on this sample dataset.

In [ ]: